KBR Internship
NIH Search Documentation

May 2026

Isabella Correa
Matthew Quintero
Maria Rodriguez Yglesias

https://github.com/izy138/KBR-search

Table of Contents

1. Project Overview

The KBR Internship NIH Project Search is a full-stack search and analytics platform built to help users explore NIH-funded research grants. It ingests multi-year (2020–2025) grant data from CSV files, indexes them into OpenSearch with optional semantic embeddings, and serves a React frontend that supports keyword search, vector similarity search, analytics dashboards, and detailed project exploration.
The platform is designed for analysts and researchers to explore NIH grant portfolios.

Users can search by keyword, filter by principal investigator, institute/center, organization, state, fiscal year range, and activity code, compare funding across activities, and discover similar projects through vector embeddings. Advanced search modal lets users build complex boolean queries with AND/OR/NOT clauses. Semantic search mode allows users to find projects by description and intent using sentence-transformer embeddings. A hybrid search mode fuses keyword and vector results using Reciprocal Rank Fusion for the best of both approaches. Users also have a dashboard page to visually explore analytics and statistics for their search, including US choropleth map, time-series charts, funding breakdowns, and interactive project term theme breakdown.

1.1 Decision Making

Cleaning Data

The raw data was provided as multiple CSV files that were missing the Abstract text column and included several columns we didn't need. To get it into a usable state, we used Pandas in a Jupyter Notebook to filter and organize each file. Once cleaned, we merged each CSV with its corresponding file containing the abstract text, joining them on the shared Application ID column.

Vectorization

To find similarities between projects, we vectorized the data, which converts text into numerical representations that can be compared mathematically. This enabled semantic search, where results are ranked by how closely they match the meaning of a query rather than just exact keywords. The practical result is that a user can select any project and see other projects ranked by how similar they are to it. Main changes: cut filler phrases, made each sentence do one job, and replaced vague wording like "discovered that vectorization can help us find data that are similar in a percentage of semantic search" with something a person can actually parse.

Parallel Prefetching

To reduce load times for the similar projects feature, we implemented parallel prefetching. As soon as a user selects a project, the app begins fetching semantically similar projects in the background, so by the time the user navigates to that section, the data is already ready. This eliminates the noticeable wait that would otherwise come from running a semantic search on demand. In the future, we plan to explore more robust caching solutions like TanStack Query or the Cache Storage API to further improve performance.

1.2 AI Tools

Claude Code was used for large audits and refactors to our project. First use was building the project's scaffolding, and planning the flow of indexing the data to open search. Another use was when we needed to redesign the dashboard page to show our graphs and visuals, Claude Code made suggestions for graphs and visuals to use. We used it to give us version 1 of our Project Term Themes section found in the dashboard, which was then iterated on. Another use was for refactoring our code base from using normal stylesheet style.css to Tailwind CSS, to give us better organization and make frontend edits easier. Lastly, Claude Code was used to give us a documentation reference which we used to write our documentation and make sure we include the necessary information. Cursor was used for general edits to the frontend and UI/UX.

2. Tech Stack

The application is built on three core services:

- React single-page application for the frontend
- Python FastAPI server for the backend API
- OpenSearch instance for full-text and vector semantic search

All three run as Docker Compose services for local development, with hot-reload enabled on both the frontend (Vite HMR) and backend (Uvicorn reload). The table below summarizes every major technology in the stack, its version, the port it exposes locally, and the role it plays in the system.

Technology	Version	Port	Role
React + TypeScript + Vite	18.3 5.9 7.1	5173	Single-page application with type safety and fast HMR
Tailwind CSS	4.3	—	Utility-first CSS; all components styled with Tailwind classes exclusively
react-router-dom	7.15	—	Client-side routing with URL state sync for shareable search links
Recharts	3.8	—	Bar, line, and pie chart components for the analytics dashboard
react-simple-maps + d3-scale	3.0 / 4.0	—	US state choropleth map with continuous color scales
Python + FastAPI + Uvicorn	3.12 0.136 0.46	8000	REST API with auto-generated OpenAPI docs at /docs
OpenSearch	2.14.0	9200	Full-text BM25 search and k-NN vector similarity
sentence-transformers (all-MiniLM-L6-v2)	5.4	—	384-dimensional sentence embeddings for semantic search
PyTorch	≥2.6	—	Tensor computation backend for embedding model inference
pandas + DuckDB	3.0 / 1.2	—	CSV loading and fast SQL filtering for CSV export
Docker Compose	—	—	Multi-service container orchestration for local development

2.1 Key Backend Python Dependencies

The backend's Python dependencies are defined in backend/requirements.txt. The most significant ones are listed here with their purpose in the system:

Package	Version	Purpose
fastapi	0.136.1	Web framework for the REST API
uvicorn[standard]	0.46.0	ASGI server with hot-reload support
pandas	3.0.2	CSV loading and data manipulation
duckdb	1.2.2	Fast SQL filtering used by CSV export
openpyxl	3.1.5	Excel file support for data loading
opensearch-py	3.2.0	OpenSearch Python client
python-dotenv	1.2.2	Environment variable loading from .env files
torch	2.6.0	PyTorch tensor backend for embedding model
sentence-transformers	5.4.1	Sentence embedding model loading and inference
jupyterlab	4.4.3	Notebook environment for exploratory analysis

2.2 Key Frontend Dependencies

The frontend's dependencies are defined in frontend/package.json. Runtime dependencies are installed in production builds, while dev dependencies (Tailwind, TypeScript, Vite) are only needed during development:

Package	Version	Purpose
react / react-dom	^18.3.1	Core UI framework
react-router-dom	^7.15.0	Client-side routing
recharts	^3.8.1	Chart components (bar, line, pie)
react-simple-maps	^3.0.0	US choropleth map rendering
d3-scale	^4.0.2	Continuous color scales for the map
react-slider	^2.0.6	Dual-handle fiscal year range slider
tailwindcss	^4.3.0	Utility-first CSS framework (dev dependency)
typescript	^5.9.3	Static type checking (dev dependency)
vite	^7.1.11	Dev server and production bundler (dev dependency)

3. System Architecture

The Three-Service Model

The platform is composed of exactly three services:

- **Frontend (port 5173) -** React + Vite frontend - a single-page application served on port 5173. The browser loads this once and all subsequent navigation is client-side.
- **Backend (port 8000) -** Python FastAPI backend - the REST API on port 8000. It receives requests from the browser, translates them into OpenSearch queries, and returns shaped JSON.
- **OpenSearch (port 9200) -** OpenSearch - the search and storage engine on port 9200. It holds all indexed grant data and executes BM25 keyword queries, k-NN vector queries, and hybrid combinations of both.

No other services are needed. The embedding model, in particular, is not a separate service as it runs inside the FastAPI process itself.

Request Flow

- The browser sends a fetch() request to the FastAPI backend at port 8000.
- FastAPI translates the request into an OpenSearch query - BM25 for keyword search, k-NN for semantic/vector search, or a Reciprocal Rank Fusion (RRF) hybrid of both.
- For semantic or hybrid requests, FastAPI first embeds the query text using the sentence-transformers model (loaded in-process) before querying OpenSearch.
- OpenSearch returns scored hits from the project_data index.
- FastAPI shapes the results into the JSON contract the frontend expects and returns them to the browser.
- The React SPA renders the results without a page reload.

Embedding Model

The embedding model runs inside the FastAPI process, loaded lazily on the first request that requires embeddings (semantic search, hybrid search, or indexing with --with-embeddings).

The model is initialized using a thread-safe singleton pattern behind a threading.Lock(), so concurrent requests don't trigger multiple loads. Once loaded, the same model instance is reused for the lifetime of the process. Device selection follows an automatic fallback chain: CUDA if available, then Apple MPS, then CPU. This can be overridden with the EMBEDDING_DEVICE environment variable. On CPU, PyTorch's intra-op thread count is capped to 4 (configurable via TORCH_NUM_THREADS) to prevent cache thrashing inside Docker Desktop's VM. In order to quickly embed the data, a separate machine with a dedicated CUDA GPU was used to reduce embedding time from hours to minutes.

3.1 Docker Compose Services

The three services are defined in docker-compose.yml:

Service	Image / Build	Ports	Key Configuration
opensearch	opensearchproject/opensearch:2.14.0	9200, 9600	Single-node, security disabled, 2 GB Java heap, health check every 10 s
backend	./backend (Python 3.12-slim)	8000	Depends on opensearch (healthy); source code and CSV files mounted for hot-reload
frontend	./frontend (Node 20-alpine)	5173	Depends on backend; npm ci && npm run dev with file polling enabled for Docker

The Opensearch service runs in single-node discovery mode with the security plugin disabled (DISABLE_SECURITY_PLUGIN=true), which is appropriate for local development but would need to be re-enabled for any production deployment. The Java heap is set to 2 GB (-Xms2g -Xmx2g), and Docker Desktop must allocate at least 4 GB of memory for this to run reliably.

3.2 Volume Mounts

Docker Compose defines two named volumes and several bind mounts.

Volume / Mount	Type	Path in Container	Purpose
opensearch-data	Named volume	/usr/share/opensearch/data	Persists the index across container restarts without needing to re-indexing after docker compose down
huggingface-cache	Named volume	/root/.cache/huggingface	Caches downloaded model weights (~90 MB for all-MiniLM-L6-v2) so they are not re-downloaded on each startup
./backend:/app	Bind mount	/app	Backend source code - enables Uvicorn hot-reload during development
./frontend:/app	Bind mount	/app	Frontend source code - enables Vite HMR during development
<year>_data.csv (×6)	Bind mount (read-only)	/app/<year>_data.csv	CSV grant data files (2020–2025) mounted into the backend for indexing and CSV export

4. Data Pipeline

The data pipeline is a set of Python scripts that transform NIH CSV files into a searchable OpenSearch index with optional semantic embeddings. The pipeline supports four operational workflows: exploratory data analysis, full bulk indexing, incremental upserts of new fiscal year data, and distributed embedding computation across machines.

4.1 Source Data and Fiscal Years

The source data files span fiscal years 2020 through 2025. Each year's CSV is mounted separately in the Docker Compose configuration and can be indexed individually (using index_data.py or append_data.py) or all at once via reindex.py.

CSV Loading (load_data.py)

1. load_data.py provides the data-reading layer used by every other script in the pipeline. Its key function is iter_csv_chunks, which streams a CSV in configurable chunks (default 2,000 rows) without loading the full file into memory.
2. Each chunk passes through _sanitize_dataframe, which converts pandas NaN and NaT values to Python None in a single vectorized pass. This is required because OpenSearch cannot parse NaN tokens in JSON payloads.
3. The module also provides load_csv (load the full file into memory as a list of dicts) and load_excel (same for .xlsx files, for cases where data arrives in Excel format).

4.2 EDA

Before the indexing pipeline was built, the raw NIH CSV data was explored using a Jupyter notebook (proj_data_analysis.ipynb) running inside the backend container via JupyterLab.

The EDA process covered: inspecting row and column counts, identifying null rates and outlier values, building correlation heatmaps and funding distribution charts, and determining which columns were useful vs. noise. This informed decisions like which fields to boost in the search query, which fields to include in embedding text construction, and how to handle recurring multi-year awards. The notebook also contains the embedding→theme assignment cell that was later extracted into build_project_term_theme_counts.py.

4.3 Indexed Fields

All 46 columns from the source CSV files are indexed. The APPLICATION_ID field serves as the deterministic OpenSearch document _id. Re-indexing the same row overwrites the existing document rather than creating a duplicate.

Field	Field	Field
APPLICATION_ID	ACTIVITY	ADMINISTERING_IC
APPLICATION_TYPE	ARRA_FUNDED	AWARD_NOTICE_DATE
BUDGET_START	BUDGET_END	ASSISTANCE_LISTING_NUMBER
CORE_PROJECT_NUM	ED_INST_TYPE	OPPORTUNITY_NUMBER
FULL_PROJECT_NUM	FUNDING_ICs	FUNDING_MECHANISM
FY	IC_NAME	NIH_SPENDING_CATS
ORG_CITY	ORG_COUNTRY	ORG_DEPT
ORG_DISTRICT	ORG_DUNS	ORG_FIPS
ORG_IPF_CODE	ORG_NAME	ORG_STATE
ORG_ZIPCODE	PHR	PI_IDS
PI_NAMEs	PROGRAM_OFFICER_NAME	PROJECT_START
PROJECT_END	PROJECT_TERMS	PROJECT_TITLE
SERIAL_NUMBER	STUDY_SECTION	STUDY_SECTION_NAME
SUBPROJECT_ID	SUFFIX	SUPPORT_YEAR
DIRECT_COST_AMT	INDIRECT_COST_AMT	TOTAL_COST
TOTAL_COST_SUB_PROJECT		

Of these 46 fields, the following are the most significant to the application's functionality:

Field	Role in the Application
PROJECT_TITLE	Primary search field (4× boost in multi_match). Displayed as the grant title throughout the UI.
PROJECT_TERMS	MeSH-style keyword list (2× boost). Used for project term filtering, theme cloud, and embedding text construction.
PI_NAMEs	Principal investigator name(s) (2× boost). Used for investigator search and PI filter.
ABSTRACT_TEXT	Grant abstract. Included in embedding text construction (truncated to 12,000 chars by default) and displayed on the Project Details page.
TOTAL_COST	Total award amount. Primary funding field for analytics charts and funding comparisons.
CORE_PROJECT_NUM	NIH core project number. Used for multi-year grant recurrence detection - groups fiscal year instances of the same award.
APPLICATION_ID	Unique application identifier. Serves as the deterministic OpenSearch document _id, making upserts idempotent.
FY	Fiscal year of the award. Used for time-series analytics, range filtering (fy_min/fy_max), and sorting.
ACTIVITY	NIH activity code (e.g. R01, R21). Used for activity filter and funding breakdown charts.
IC_NAME	NIH Institute or Center name. Used for IC filter and by-IC analytics aggregation.
ORG_NAME	Organization/institution name. Used for organization filter and top-orgs analytics.
ORG_STATE	US state of the organization. Used for state filter and US choropleth map.

4.4 OpenSearch Indexing

All grant data is stored in a single OpenSearch index called project_data. The index name is configurable via the OPENSEARCH_INDEX environment variable.

Dynamic Mapping

The index uses dynamic mapping by default. OpenSearch infers field types from the CSV values at index time - strings become text fields with keyword sub-fields, numbers become long or float, and so on. No explicit schema is defined for the 46 CSV columns.

The one exception is the embedding field (see below), which must be declared explicitly before any vectors are written.

Embedding Field Mapping

When data is indexed with the --with-embeddings flag, the index is created with KNN enabled and an explicit knn_vector mapping for the embedding field:

```
{"embedding": {"type": "knn_vector", "dimension": 384,
  "method": {"name": "hnsw", "engine": "lucene", "space_type":
"cosinesimil"}}}
```

The key design choices in this mapping:

- 384 dimensions - matches the output dimension of the all-MiniLM-L6-v2 sentence-transformers model. Indexing and querying must use the same model; a dimension mismatch raises a 409 error at query time.
- HNSW algorithm - Hierarchical Navigable Small World provides approximate nearest-neighbor search with sub-linear query time. It trades a small amount of recall for very fast lookups.
- Lucene engine - used instead of nmslib because it integrates natively with OpenSearch's segment structure and requires no separate native library.
- cosinesimil space type - appropriate because the sentence-transformers model produces L2-normalized vectors. Cosine similarity on normalized vectors is mathematically equivalent to dot product.

Security

OpenSearch's security plugin is disabled for local development (DISABLE_SECURITY_PLUGIN=true in docker-compose.yml). The opensearch_client.py connection is configured with use_ssl=False, verify_certs=False, and ssl_show_warn=False. All communication between the backend and OpenSearch happens over unencrypted HTTP on port 9200.

For any deployment beyond a local development machine, the security plugin must be re-enabled and SSL configured.

4.5 Pipeline Overview

The data pipeline is a set of focused Python scripts, each responsible for one stage. They share common building blocks (load_data.py for CSV reading, index_data.py's bulk_load for writing to OpenSearch) so behavior is consistent across workflows.

	Script	Responsibility
1	`load_data.py`	Reads the CSV in 2,000-row chunks; sanitizes NaN/NaT to null
2	`index_data.py`	Optionally attaches embeddings (_attach_embeddings), generates bulk actions with APPLICATION_ID as _id, ships to OpenSearch via parallel_bulk (8 threads, 1,000-doc batches, gzip)
3	`reindex.py`	Drops the existing index, recreates it with correct mappings, then runs the full index_data.py pipeline from scratch
4	`append_data.py`	Upserts a new CSV into the existing index - no rebuild required; reuses index_data.py bulk_load
5	`export_embeddings.py`	GPU machine: reads a CSV, computes embeddings, writes an NDJSON file
6	`import_embeddings.py`	Indexing machine: reads NDJSON files from export step and bulk-indexes with KNN mapping
7	`term_stats.py`	Computes per-term document frequencies across all CSVs; outputs term_stats.json used to filter generic terms before embedding
8	`build_project_term_theme_counts.py`	Assigns PROJECT_TERMS tokens to theme categories by cosine similarity; outputs project_term_theme_counts.json for the dashboard word cloud

The four main operational workflows are:

1. Analytics precomputation - term_state.py and build_project_term_theme_counts.py generates static JSON files for keyword sections used by the API and dashboard. Run once per corpus refresh.
2. Full reindex - delete the index, recreate with the right mappings, bulk-load all CSV years. Use when setting up a new environment or changing index mappings.
3. Incremental upsert - append_data.py adds a new year's CSV to the existing index without a rebuild.
4. Distributed embedding - export_embeddings.py runs on a GPU machine to generate vectors; import_embeddings.py loads the resulting NDJSON files on the indexing machine.

Term Statistics (term_stats.py)

term_stats.py computes document frequencies for every term in the PROJECT_TERMS field across one or more CSV files. Its output, term_stats.json, tells the embedding system which terms are too common to carry useful signals in vector representations.

The output file contains:

- doc_count - total documents scanned across all input files.
- terms array - sorted by descending document frequency. Each entry includes the term string, df (raw document frequency), df_ratio (df / doc_count), and idf (log-smoothed inverse document frequency).

The embedding system reads term_stats.json at startup and strips any term with df_ratio ≥ 0.20 before encoding. Terms like "research", "data", and "human" appear in more than 20% of all projects and add noise to the embedding without distinguishing one grant from another. Removing them produces denser, more discriminative vectors.

Run term_stats.py once per refresh (i.e., whenever new CSV years are added). If term_stats.json does not exist, the system proceeds without filtering - no terms are stripped.

```
docker compose exec backend python indexer/term_stats.py \
     /app/2020_data.csv /app/2021_data.csv ... /app/2025_data.csv
```

Term Theme Taxonomy (build_project_term_theme_counts.py)

build_project_term_theme_counts.py precomputes the hierarchical theme assignments that power the dashboard's term theme cloud visualization. It is the production version of the embedding - theme assignment cell from the Jupyter notebook.

1. Streams PROJECT_TERMS tokens from all CSV files and tallies their corpus-wide frequencies.
2. Loads the sentence-transformer model and encodes both the term tokens and a set of predefined category anchor paragraphs (one per theme subcategory).
3. Assigns each token to its nearest subcategory anchor by cosine similarity.
4. Writes project_term_theme_counts.json with both a flat buckets array (label + weight) and a nested tree structure for the frontend.

The output is organized into six top-level theme categories: Health, Life Sciences, Computing & AI, Environmental, Engineering, and Education & Social Sciences. The frontend uses the flat buckets for the word cloud and the tree for the interactive drill-down browser.

Run this script once after indexing, and re-run it whenever the corpus changes significantly. The API reads the output file statically at the /analytics/project-term-theme-cloud endpoint - no OpenSearch query is involved.

```
docker compose exec backend python
     indexer/build_project_term_theme_counts.py
```

Vector Embeddings

A separate machine with a dedicated GPU reduced total embedding time from approximately 12 hours (CPU) to 20 minutes. The export/import workflow lets you generate vectors on a GPU machine and transfer them separately, without needing GPU access on the indexing machine. These files are transferred to the machine you want to import data locally to.

export_embeddings.py (runs best on GPU machine)

Reads a CSV, computes embeddings using the same model and text preprocessing as the indexer (build_text_for_record from embeddings.py - PROJECT_TITLE + filtered PROJECT_TERMS + truncated ABSTRACT_TEXT), and writes an NDJSON file. Each line of the NDJSON is a complete JSON object containing all document fields plus the embedding vector as a JSON array.

import_embeddings.py (runs on any machine)

Reads one or more NDJSON files produced by the export step and bulk-indexes them using the same parallel_bulk settings as index_data.py. Re-importing the same file is a safe idempotent upsert If the index does not exist, it is created with KNN mapping automatically.

⚠ Indexing and embedding must use the same model. Vectors from different models live in different geometric spaces and cannot be compared. The project standard is sentence-transformers/all-MiniLM-L6-v2 (384-d).

Bulk Indexing (index_data.py)

index_data.py is the core indexing script. It reads chunks from load_data.py and writes them to OpenSearch via a four-stage pipeline:

1. iter_csv_chunks - reads the CSV in 2,000-row chunks with NaN sanitized.
2. _attach_embeddings - (optional, --with-embeddings flag) encodes each chunk's text in a single batched call to the sentence-transformers model. Batching is important as encoding the full chunk in one call is significantly faster than per-row encoding because the model amortizes overhead across the batch.
3. _actions - generates OpenSearch bulk action dictionaries. APPLICATION_ID is used as the deterministic _id, making reruns safe - the same row always overwrites the same document.
4. parallel_bulk - sends actions to OpenSearch across 8 worker threads in 1,000-document batches with gzip compression. Failures are logged but do not abort the run, so a single malformed row never kills a large ingest.

During a bulk load, the script temporarily relaxes two OpenSearch index settings to maximize write throughput:

- refresh_interval: -1 disables automatic segment refresh so OpenSearch is not building searchable segments after every batch.
- number_of_replicas: 0 disables replication during the load.

Both settings are restored to their original values after the load completes, and a manual index refresh is triggered via a context manager (_bulk_load_settings). This pattern is a standard OpenSearch bulk-ingest optimization.

Wipe & Rebuild (reindex.py)

reindex.py performs a full index rebuild: it deletes the existing index if it exists, recreates it with the correct mappings (including KNN if --with-embeddings is passed), then runs the full bulk_load pipeline from index_data.py.

Use this when:

- Setting up a fresh environment with no existing index.
- Changing index mappings - for example, enabling KNN on an index that was originally created without it.
- Recovering from a corrupted or incomplete index.

⚠ reindex.py destroys all existing data. Use append_data.py for adding new year data to an existing index.

Incremental Load (append_data.py)

append_data.py safely upserts a new CSV into an existing index without a full rebuild. It reuses the same bulk_load and create_index functions from index_data.py. The create_index call verifies that the existing index has the required embedding mapping if --with-embeddings is passed.

Because APPLICATION_ID is used as the deterministic document _id, overlapping rows are updated in place rather than duplicated. This is the standard workflow for adding a new fiscal year: the existing index stays intact, and only new or changed rows are written.

5. Backend API References

The API is organized into two routers:

- Search router (/search) - handles keyword search, project lookup, similar project discovery, hybrid search, investigator queries, and CSV export.
- Analytics router (/analytics) - handles dashboard aggregations, chart data, and term taxonomy endpoints.

5.1 Common Filter Parameters

The following parameters are accepted by GET /search/ and all analytics aggregation endpoints. They are implemented as non-scoring filter clauses applied to the OpenSearch bool query.

Param	Type	Default	Description
`pi`	str	`""`	PI name filter (phrase match or token-AND on PI_NAMEs)
`ic`	str	`""`	Exact match on IC_NAME.keyword
`activity`	str	`""`	Exact match on ACTIVITY.keyword
`state`	str	`""`	Exact match on ORG_STATE.keyword
`fy_min`	int|None	`None`	Minimum fiscal year
`fy_max`	int|None	`None`	Maximum fiscal year

The principal investigator parameter uses a bool/should with phrase match (match_phrase) and token-AND match (match with operator: and), minimum_should_match: 1. This handles both "Last, First" and "First Last" name formats. All other filters use exact keyword term matching or range queries.

5.2 Health Check

GET /health

Returns the API status and whether OpenSearch is reachable. Takes no parameters and never errors, it is designed for container health checks and uptime monitoring.

- Response (healthy): { "status": "ok", "opensearch": "up" }
- Response (degraded): { "status": "ok", "opensearch": "down" }

5.3 Search Endpoints

GET /search/

The primary keyword search endpoint. Uses OpenSearch's multi_match query with best_fields type and AND operator across six weighted fields:

- PROJECT_TITLE - 4× boost
- ABSTRACT_TEXT, PROJECT_TERMS, PI_NAMEs - 2× boost each
- ORG_NAME, IC_NAME, ACTIVITY - 1× boost (no boost)

When q is empty and no advanced_q is provided, the query falls back to match_all and returns all documents. The project_terms filter applies an additional token-AND match on PROJECT_TERMS for each term provided (all terms must match - AND across terms).

Param	Type	Default	Constraints	Description
`q (query)`	str	`""`	max 1,000 chars	Search query across the six weighted fields
`advanced_q`	str	`""`	Valid JSON	Boolean query: {"clauses":[{text,negated}],"operators":["and"|"or"]}
`limit`	int	`10`	1–100	Results per page
`page`	int	`1`	≥1	1-based page index; offset must be < 10,000
`category`	str	`""`	—	Exact match on category.keyword
`pi (principal investigator)`	str	`""`	—	PI name filter (phrase match or token-AND on PI_NAMEs)
`ic (Institute / Center)`	str	`""`	—	Exact match on IC_NAME.keyword
`activity`	str	`""`	—	Exact match on ACTIVITY.keyword
`state`	str	`""`	—	Exact match on ORG_STATE.keyword
`fy_min`	int|None	`None`	—	Minimum fiscal year (range gte)
`fy_max`	int|None	`None`	—	Maximum fiscal year (range lte)
`project_terms`	list[str]	`[]`	max 20, 200 chars each	Include terms: all tokens in each term must appear in PROJECT_TERMS (AND across terms)

`exclude_project_terms`	list[str]	`[]`	max 20, 200 chars each	Exclude terms: projects matching any of these terms are removed (AND across phrases)
`sort_by`	str	`""`	See note	Sort field: PROJECT_TITLE, PI_NAMEs, ORG_NAME, IC_NAME, ORG_STATE, ACTIVITY, FY, or TOTAL_COST
`sort_order`	str	`"asc"`	asc | desc	Sort direction

Response fields: total (true hit count from OpenSearch), visible_total (capped at 10,000 pagination window limit), results (array of project documents, each with _id and optionally _score).

GET /search/export

Exports search results as a downloadable CSV file enriched with all original columns from the source CSV files. Accepts all the same filter parameters as GET /search/, plus:

Param	Type	Default	Constraints	Description
`max_rows`	int	`10000`	1–10,000	Maximum number of rows to include in the export

The endpoint scrolls OpenSearch to collect matching APPLICATION_ID and FY keys, then joins them against the original CSV files using DuckDB for fast SQL-based filtering (with a pandas fallback if DuckDB is unavailable). The response filename is derived from the query slug.

GET /search/project/{project_id}

Fetches a single project document by its OpenSearch _id.

Returns { "project": { ...all fields... } } on success.

Param	Type	Default	Constraints	Description
`project_id`	str (path)	`required`	—	The OpenSearch document _id for the project

GET /search/project/{project_id}/other-years

Finds other fiscal year instances of the same recurring NIH award. Looks up the project, then searches for siblings by CORE_PROJECT_NUM. If that only returns one result (the project itself), it falls back to matching by PROJECT_TITLE.

The response includes both a years array (all fiscal years including the current one) and an other_years array (excluding the current project). Each entry contains project_id, application_id, fy, and is_current.

Param	Type	Default	Constraints	Description
`project_id`	str (path)	`required`	—	The OpenSearch document _id for the project

⚠ Returns 404 if the project is not found. Returns 400 if the project has neither CORE_PROJECT_NUM nor PROJECT_TITLE.

GET /search/similar

Performs vector similarity search by embedding the query text at request time and returning the nearest neighbors by cosine similarity. The same sentence-transformers model used at index time must be loaded (model mismatch returns 409).

Param	Type	Default	Constraints	Description
`q`	str	`required`	min_length=1, max 1,000 chars	Free-text query to embed and search against the vector index
`k`	int	`10`	1–50	Number of nearest neighbors to return

Each result includes a _score field representing cosine similarity (0–1 range). Embedding field is excluded from the response payload.

⚠ Returns 409 if the loaded embedding model's dimension does not match the index mapping.

GET /search/similar/{project_id}

Finds projects similar to an existing indexed project using its stored embedding vector - no re-embedding required. The source document's vector is retrieved and used directly for the k-NN query.

Sibling fiscal years of the same recurring award are excluded from results using must_not clauses on CORE_PROJECT_NUM and PROJECT_TITLE. The endpoint over-fetches (k×4 or k+15, whichever is larger, capped at 50) to compensate for year deduplication. Results are grouped by recurrence key, and each group includes a year_variants array listing all available fiscal year instances of that grant.

Param	Type	Default	Constraints	Description
`project_id`	str (path)	`required`	—	The OpenSearch _id of the source project
`k`	int	`10`	1–100	Number of similar projects to return (after deduplication)

⚠ Returns 404 if the project does not exist. Returns 409 if the project has no stored embedding (indexed without --with-embeddings).

GET /search/hybrid

Hybrid search that runs BM25 keyword search and k-NN vector search in parallel, then fuses their rankings using Reciprocal Rank Fusion (RRF). Combines the precision of keyword matching with the semantic breadth of vector search.

Implementation details: both searches run concurrently via ThreadPoolExecutor. Each side fetches min(max(k×4, 25), 100) candidates. The RRF formula scores each document as the sum of 1/(60+rank) across both lists, where 60 is the standard smoothing constant from the original RRF paper. Raw BM25 and cosine scores are discarded - only rank order matters.

Param	Type	Default	Constraints	Description
`q`	str	`required`	min_length=1	Free-text query; used for both BM25 and embedding
`k`	int	`10`	1–50	Number of fused results to return

Each result includes _score (RRF fused score), _rank_keyword (rank in BM25 list, or null if absent), and _rank_vector (rank in vector list, or null if absent). The response also includes keyword_total, keyword_returned, vector_returned, and fetch_size_per_side for diagnostics.

⚠ Returns 409 if the embedding model dimension does not match the index mapping.

GET /search/investigator/{pi_name}

Returns a paginated list of all grants for a principal investigator. The query uses a bool/should with three matching strategies on PI_NAMEs (minimum_should_match: 1):

- Exact keyword term match on PI_NAMEs.keyword
- Phrase match (match_phrase)
- Token-AND match (match with operator: "and") - all tokens must appear, order-independent

This handles both "Last, First" and "First Last" input formats. Results are sorted by fiscal year descending.

Param	Type	Default	Constraints	Description
`pi_name`	str (path)	`required`	—	Principal investigator name
`limit`	int	`25`	1–100	Results per page
`page`	int	`1`	≥1	1-based page index

Response fields: investigator_name, limit, total, visible_total (capped at 10,000), results.

⚠ Returns 400 if the requested page offset exceeds 10,000.

5.4 Analytics Endpoints

All analytics endpoints accept the common filter parameters from Section 5 (pi, ic, activity, state, fy_min, fy_max) unless noted otherwise. Filters are applied as non-scoring bool filter clauses so they do not affect aggregation scoring.

GET /analytics/summary

Returns a high-level summary of the indexed data: total document count, breakdown by category, and basic time-series data. Accepts common filter parameters.

GET /analytics/by-state

Returns per-state document counts and total funding, used to populate the US choropleth map. Accepts common filter parameters.

Response: array of { state, count, total_funding }.

GET /analytics/by-ic

Returns document counts per NIH Institute/Center (IC_NAME). Accepts common filter parameters plus one additional endpoint-specific parameter:

Param	Type	Default	Constraints	Description
`fy`	Int | None	`None`	2000–2100	Optional fiscal year filter. When provided, the response shows counts only for that fiscal year rather than across all years.

Response: array of { label, value } where label is the IC name and value is the document count.

GET /analytics/by-activity

Returns document counts and total funding per activity code. Accepts common filter parameters.

Param	Type	Default	Constraints	Description
`limit`	int	`50`	1–200	Maximum number of activity codes to return

Response: array of { label, total_funding, count }.

GET /analytics/by-activity-funding-pie

Returns activity code funding data shaped for a pie chart, with optional merging of the long tail into a single "Other" slice. Accepts common filter parameters.

Param	Type	Default	Constraints	Description
`pie_slices`	int	`10`	1–50	Number of top activity codes to return as named slices
`merge_other`	bool	`true`	—	When true, activity codes beyond pie_slices are merged into a single Other slice

Response fields: total_funding_indexed, slices (named activity slices), other (merged tail slice or null), tail_slices (individual tail codes when merge_other is false), and various metadata about the aggregation.

GET /analytics/by-year

Returns document counts and total funding per fiscal year, sorted ascending. Accepts common filter parameters.

Response: array of { year, count, total_funding }.

GET /analytics/top-orgs

Returns the top 15 organizations by total funding. Accepts common filter parameters. Results are sorted by total_funding descending.

Response: array of { label, total_funding }.

GET /analytics/avg-grant-by-ic

Returns the average grant size per NIH Institute/Center, sorted by average descending. Accepts common filter parameters. Returns up to 100 ICs.

Response: array of { label, avg_grant }.

GET /analytics/by-activity-terms

Returns the top PROJECT_TERMS for a specific activity code, with document counts and total funding. Does not accept common filter parameters - filters by activity code only.

Param	Type	Default	Constraints	Description
`activity_id`	str	`required`	—	Activity code to query (e.g. R01)
`limit`	int	`25`	1–100	Maximum number of terms to return

Response: { activity_id, limit, data: [{ label, count, total_funding }] } - sorted by total_funding descending.

GET /analytics/by-activity-project-compare

Returns funding data for a selected project and its peer projects within the same activity code. Used for the bar chart comparison on the Project Details page. The selected project always appears first in the data array.

Param	Type	Default	Constraints	Description
`project_id`	str	`required`	—	OpenSearch document _id for the selected project
`activity_id`	str	`required`	—	Activity code to pull peers from (e.g. R01)
`limit`	int	`20`	1–20	Maximum number of peer projects to include

Response: { project_id, activity_id, data: [{ project_id, label, total_funding, is_selected }]} - peers sorted by total_funding descending, with the selected project prepended.

⚠ Returns 404 if the selected project is not found.

GET /analytics/project-term-theme-cloud

Returns precomputed theme bucket data for the dashboard word cloud. Data is read from the static project_term_theme_counts.json file generated by the build_project_term_theme_counts.py script. No OpenSearch query is performed.

Response: { generated_at, method, low_confidence_cosine, buckets: [{ label, weight }], source_path }. If the file does not exist, returns an empty buckets array and a message explaining how to generate it.

⚠ Returns HTTP 500 if the JSON file exists but cannot be parsed.

GET /analytics/term-tree

Returns the static 3-level NIH research term hierarchy used by the frontend term-cloud browser. The hierarchy is hardcoded - no OpenSearch lookup is performed. Leaf labels are drawn from the NIH PROJECT_TERMS vocabulary and organized into 6 top-level categories: Health, Life Sciences, Computing & AI, Environmental, Engineering, and Education & Social Sciences.

Response: array of top-level category nodes. Each node has id (dot-notation), label, and children. Leaf nodes omit children.

6. Frontend Pages & Components

6.1 Search Modes Overview

The application supports four search modes, each backed by different OpenSearch query strategies. The table below summarizes them:

Mode	Endpoint	Scoring	Filters	Notes
Keyword (BM25)	GET /search/	BM25 multi_match (4× title, 2× terms/PI)	All 6 common filters + project_terms	Default mode. Falls back to match_all when query is empty.
Advanced Boolean	GET /search/ (advanced_q param)	Same BM25 fields	All 6 common filters	Up to 8 clauses with AND/OR/NOT. Mutually exclusive with semantic mode.
Semantic	GET /search/similar?q=	Cosine similarity (k-NN)	None (query text only)	Toggled via checkbox on search page. Client-side pagination over up to 50 results.
Hybrid (RRF)	GET /search/hybrid	Reciprocal Rank Fusion of BM25 + k-NN	All 6 common filters	Dedicated page at /semantic. Rank-based fusion avoids score calibration issues.

Keyword Search is the default mode and the one most users interact with. It uses OpenSearch's multi_match query with best_fields type and the AND operator across six weighted fields (see GET /search/ in Section 5.3 for the full field list and boost values). When no query text is provided, the search falls back to match_all, returning all documents and allowing users to browse the dataset using only filters and sorting.

Advanced Boolean Search lets users build structured queries through the AdvancedSearchModal component. A query consists of up to 8 clauses, each up to 200 characters, combined with AND or OR operators and individually negatable with NOT. The frontend serializes these into a unified string format and passes them as the advanced_q JSON parameter.

Semantic Mode can be enabled via a checkbox toggle on the main search page. The toggle has a two-phase commit: checking the box sets a pending state, but the vector search does not execute until the user clicks the search button. When committed, the frontend calls GET /search/similar?q=<query>&k=50 and paginates over the results client-side. Filters are not applied in semantic mode because the /search/similar endpoint performs a pure k-NN search without filter clauses.

Hybrid Search is available on a dedicated page at /semantic. It runs BM25 and k-NN retrieval in parallel and fuses rankings with Reciprocal Rank Fusion (see GET /search/hybrid in Section 5.3 for the RRF algorithm details). Each result shows its fused score and individual BM25/vector ranks for full transparency.

All paginated endpoints enforce a MAX_RESULT_WINDOW of 10,000 results. The frontend displays visible_total (capped at 10,000) alongside the true total count so users can see that more results exist even when they cannot paginate further.

6.2 Application Shell (App.tsx)

App.tsx is the root component and the single largest file in the frontend. It owns all top-level state, handles routing, and orchestrates the interactions between the search bar, filters, results list, and detail pages. The component uses react-router-dom's BrowserRouter (set up in main.tsx with StrictMode) and lazy-loads the four heaviest route components: Dashboard, ProjectDetailsPage, and SemanticVectorLabPage.

The header renders three navigation buttons (Dashboard, Search, Hybrid Search) plus a theme toggle for light/dark mode and partner logos (KBR, FIU). Routing is handled with matchPath checks:

- if the current path is / or /dashboard the app renders the Dashboard view
- if it's /search the app renders the search page
- all other routes are handled by their respective lazy-loaded components wrapped in ErrorBoundary for crash isolation.

The state managed in App.tsx is extensive. The search state includes:

- searchQuery (the unified search input combining plain and advanced queries)
- semanticSearchMode and semanticSearchCommitted (the semantic toggle has a "pending" and "committed" state so toggling doesn't fire a search until the user clicks the search button)
- advancedSearch (a structured AdvancedSearchQuery object)
- selectedPI, selectedIC, selectedActivity, selectedState, fyMin, fyMax - individual filter field values
- resultsPerPage, currentPage, sortOption, columnSort - pagination and sort state
- investigatorPage, organizationPage, institutionPage - separate per-page counters for each entity listing page

6.3 Dashboard

The Dashboard component fetches data from nine analytics endpoints in parallel on mount and whenever filters change. Filter state is converted to API parameters via toAnalyticsFilterOptions(), which is memoized to prevent unnecessary refetches when filter values haven't actually changed.

At the top of the dashboard, three KPI (Key Performance Indicator) cards display:

- Total Funding (formatted with formatDollarsCompact),
- Total Projects (locale-formatted count),
- Average Grant (total_funding divided by total_documents).

Below these, the dashboard renders a grid of chart components:

Chart	Component	API Endpoint	Description
State Map	StateMap	/analytics/by-state	US choropleth map colored by project count. Allows users to click a state to filter
Institute Projects	BarChartPanel	/analytics/by-ic	Horizontal bars with log/linear toggle and hybrid axis scale
Year Trends	LineChartPanel	/analytics/by-year	Dual-axis: project count (left, blue) + funding (right, green)
Activity Pie	ActivityFundingPiePanel	/analytics/by-activity-funding-pie	Two-ring donut: top slices on the inner circle, remaining values on the outer
Top Orgs	BarChartPanel	/analytics/top-orgs	Horizontal bars ranked by total funding
Avg Grant by IC	BarChartPanel	/analytics/avg-grant-by-ic	Average grant size per institute/center
Term Themes	ProjectTermsThemeCloud	/analytics/project-term-theme-cloud	Interactive hierarchical term visualization

6.4 Search Page (/search)

The search page combines six components: SearchBar, FilterField, FilterSelect, FiscalYearRangeSlider, AdvancedSearchMode, and ResultsList. All state is driven by URL parameters via useSearchUrlSync, which maintains bidirectional sync between URL search params and component state. This means refreshing the browser preserves the current search, and search URLs are shareable.

The SearchBar component supports three modes that are mutually exclusive. In plain search mode, the user types a keyword query. In advanced search mode (toggled via a button), an AdvancedSearchModal opens for building boolean queries with up to 8 clauses, each combinable with AND/OR operators and individually negatable with NOT. In semantic mode (toggled via a checkbox), the search uses pure vector similarity.

The Filters panel provides six filter fields:

- Principal Investigator (text input that commits on blur or Enter)
- Institute (dropdown select with formatted names)
- Organization (dropdown select)
- Activity Code (dropdown)
- State (dropdown)
- Fiscal Year (a dual-handle range slider with text inputs).

Select fields apply immediately, while the PI text input and fiscal year slider use a draft pattern - changes accumulate locally and commit on blur, Enter, or drag release.

Column headers are clickable for sorting. The sort cycles through three states: none → ascending → descending → none. Column sort takes precedence over the dropdown sort selector, which offers “Most Relevant,” “Title: A to Z,” and “Title: Z to A.”

6.5 Project Detail Page (/projects/:projectId)

The project detail page uses a two-column layout. The left column contains the header (back button + fiscal year tags for this project's year variant), the full title, an expandable abstract (truncated at 1,500 characters with a "Read more"/"Show less" toggle), and an 8-field metadata grid: PI names, organization, location, NIH institute/center, fiscal years, project dates, award amount, and activity code. A familyYearsCacheRef caches this project's year-variant data so the header tags don't flicker when navigating between fiscal years of the same project.

Below the metadata is an interactive keyword tag panel. The project's PROJECT_TERMS are displayed as clickable tags with a three-state cycle: unselected → include (blue) → exclude (red) → unselected. Selected tags can be used to search for other projects with those terms. Below this, the ProjectSimilarProjectsChart shows a horizontal bar chart comparing the current project's award amount against up to 10 peer projects with the same activity code, using data from GET /analytics/by-activity-project-compare.

The right sidebar displays the top 10 similar grants via the semantic similarity endpoint (GET /search/similar/{id}). Each similar project shows its title, activity badge, award amount, and fiscal year variant tags. A "See more similar projects" link navigates to /semantic/similar/{id} for the full-page view. The sidebar also includes smart caching of fiscal year family data (via familyYearsCacheRef) to preserve year information when navigating between similar projects without triggering redundant API calls.

6.6 Data Fetching

On mount, the component fires two API calls in parallel using Promise.allSettled():

- GET /search/project/{id} via getProjectById() - fetches the anchor/reference project's full record
- GET /search/similar/{id}?k=100 via searchSimilarToProjectId() - fetches up to 100 nearest neighbors by cosine similarity from the k-NN index

Using Promise.allSettled() means failures in one call do not block the other. The component maintains separate error states for the anchor project and the similar results, allowing partial rendering when one succeeds and the other fails.

6.7 Deduplication of Results

The raw k-NN results often contain multiple fiscal-year variants of the same underlying project. Before rendering, results are piped through groupSimilarNeighbors() from utils/recurrenceGrouping.ts, which collapses these duplicates into a single row with an aggregated year_variants array. The grouping logic works as follows:

- Primary grouping key: normalized PROJECT_TITLE (trimmed, lowercased, whitespace-collapsed)
- Fallback grouping key: normalized CORE_PROJECT_NUM
- Last resort: the document _id
- Score retention: when multiple fiscal-year versions merge, the version with the highest similarity score (_score) is kept as the representative record
- Year variants: all unique fiscal year / project ID pairs are collected, deduplicated by (fy, project_id), and sorted by fiscal year ascending

A secondary merge pass checks for records that share the same CORE_PROJECT_NUM but had different titles (and therefore different primary keys). This catches edge cases where the same project had slightly different titles across years.

6.8 Similarity Score Column

The ResultsList component is rendered with the showSimilarityScore prop set to true. This inserts a "Score" column before the Principal Investigator column, displaying the cosine similarity as a 4-decimal monospace value (e.g., 0.8234). The column is sortable via client-side sorting since all results are loaded at once (no server-side pagination). By default, results are ordered by descending similarity score (the order returned by the API).

6.9 Shared Entity Page Pattern

The Investigator, Organization, and Institution pages all follow the same architectural pattern: a dedicated data-fetching hook (useInvestigatorProjects, useOrganizationProjects, useInstitutionProjects) feeds data into the shared InvestigatorPage rendering component. Each hook calls its own API endpoint but manages identical state: results, loading, error, total, visibleTotal. App.tsx orchestrates which hook and route are active via matchPath checks, and maintains separate pagination state (currentPage) for each entity type so that navigating between entities preserves each page's scroll position.

Investigator Page (/investigators/:name)

The investigator page is relatively straightforward: it displays all grants for a principal investigator with pagination (25 per page). Data is fetched via the useInvestigatorProjects hook, which calls GET /search/investigator/{name}. The page

reuses the same ResultsList component as the search page with primarySort set to “relevant,” and includes a “Back to results” button for navigation. The header shows the investigator name, project count, and current page information.

Organization Page (/organizations/:organizationName)

The Organization Page displays all grants associated with a specific research organization (university, hospital, research institute, etc.). Users navigate to this page by clicking an organization name in the ORG_NAME column of any ResultsList table. The route is /organizations/:organizationName, where the name is URL-encoded.

Institution Page (/institutions/:institutionName)

The Institution Page displays all grants funded by a specific NIH institute or center (IC). Users navigate to this page by clicking an institution name in the IC_NAME column of any ResultsList table. The route is /institutions/:institutionName, where the name is URL-encoded.

6.10 Hybrid Search Page (/semantic)

The Hybrid Search page is rendered by SemanticVectorLabPage.tsx, a lazy-loaded component accessible from the "Hybrid Search" tab in the main navigation header. It provides a dedicated interface for running hybrid keyword + vector searches that merge BM25 text matching and k-NN vector similarity using Reciprocal Rank Fusion (RRF). The API endpoint is GET /search/hybrid, called via the searchHybrid() function in api.ts.

How Hybrid Search Works

The backend runs two retrieval strategies in parallel: BM25 (keyword matching) and k-NN (vector similarity using sentence-transformer embeddings). The two ranked lists are merged using Reciprocal Rank Fusion, which scores each document as the sum of 1/(k + rank) across the lists it appears in. The filters (activity, state, fiscal year) apply to both sides of the search simultaneously.

6.11 Error Handling

The page handles several error scenarios gracefully:

- Anchor project not found: "No project exists with this id, or the API is unreachable."
- Similarity search failed (embedding missing): displays a hint: "This index row has no stored vector. Re-import NDJSON embeddings or reindex with embeddings so k-NN can run."
- No neighbors returned: "No neighbors returned (unexpected for a valid embedded project)."

7. Local Setup How To

This guide covers everything needed to get the KBR NIH Project Search running locally from scratch. The stack consists of three services — **OpenSearch** (search engine), **FastAPI backend**, and a **React/Vite frontend** — all orchestrated by Docker Compose. **Docker is the only supported way to run this project.** The frontend and backend must not be started outside of Docker.

1. Prerequisites

The only tools you need on your machine before starting are:

Tool	Minimum Version	Notes
Git	2.x	For cloning the repository
Docker Desktop	4.x	Required to run all services

> **Memory requirement:** OpenSearch needs at least **4 GB of RAM** allocated to Docker Desktop. Go to Docker Desktop → Settings → Resources → Memory and set it to 4 GB or higher. If skipped, OpenSearch will crash on startup.

2. Clone the Repository

```
git clone https://github.com/izy138/KBR-Internship.git
cd KBR-Internship
```

3. Add the Data Files

Data files are **not included in the repository** (gitignored due to size). Obtain them from the team.

> **Vector embeddings are required.** Loading data always goes through the *_embedded.ndjson files produced by export_embeddings.py (or supplied pre-built by the team). Do ***not*** index plain CSVs without embeddings — documents will lack vectors and semantic/similar search will not work.

3.1 What You Need

File Pattern	Required For
2020_data.csv … 2025_data.csv	Generating embedded NDJSON (Step 4a) — only needed if you are producing NDJSON files yourself
2020_data_embedded.ndjson … 2025_data_embedded.ndjson	Indexing into OpenSearch (Step 4b) — required for the app to work
2020_PROJECT.csv … 2025_PROJECT.csv	Download CSV functionality

3.2 Where to Put Them

Place files inside the backend/ directory, under indexer/data/ for indexed data and indexer/OGdata/ for the original export CSVs used for the download feature:

```
KBR-Internship/
└── backend/
    └── indexer/
        ├── data/              ← Index & semantic search files
        │   ├── 2020_data.csv
        │   ├── 2020_data_embedded.ndjson
        │   ├── ...
        │   ├── 2025_data.csv
        │   └── 2025_data_embedded.ndjson
        └── OGdata/            ← Download CSV functionality
            ├── 2020_PROJECT.csv
            ├── ...
            └── 2025_PROJECT.csv
```

Inside the backend container these files appear at /app/indexer/data/ and /app/indexer/OGdata/ respectively (the entire backend/ folder is mounted at /app).

> You can start with a single year (e.g. 2025_data_embedded.ndjson) and add more later. Process one year at a time.

4. Start the Services

From the project root, run:

```
docker compose up --build
```

This starts three services:

Service	URL	Description
OpenSearch	http://localhost:9200	Search engine — takes 15–30 seconds to become healthy
Backend	http://localhost:8000	FastAPI server — waits for OpenSearch before starting
Frontend	http://localhost:5173	React/Vite dev server — waits for backend before starting

Wait until all three services are running and you see no error messages. On first startup the frontend runs npm ci, which may take a minute.

4.1 Verify OpenSearch

```
curl http://localhost:9200
```

You should see a JSON response containing "status" : "green" or "status" : "yellow".

4.2 Verify the Backend

```
curl http://localhost:8000/health
```

Expected response: {"status":"ok","opensearch":"up"}

5. Load Data into OpenSearch

The database starts empty. Every record must be indexed **with** a pre-computed embedding vector. The required flow is:

1. **Export —** encode CSV rows and write NDJSON (export_embeddings.py)
2. **Import —** bulk-load NDJSON into OpenSearch (import_embeddings.py)

If the team already provided *_embedded.ndjson files, skip to Step 5b.

5a. Generate Embedded NDJSON from CSV

> Skip this step if you already have the .ndjson files in your data folder. This step is only needed when adding new years of data or if you need to regenerate the NDJSON files from scratch.

Pipeline Order:

term_stats.py → term_stats.json → export_embeddings.py → *_embedded.ndjson → import_embeddings.py

Term stats MUST be computed before exporting embeddings:

Single Year:

```
docker compose exec backend python indexer/term_stats.py
/app/indexer/data/2025_data.csv
```

Multi Year:

```
docker compose exec backend python indexer/term_stats.py \
  /app/indexer/data/2020_data.csv \
  /app/indexer/data/2021_data.csv \
  /app/indexer/data/2022_data.csv \
  /app/indexer/data/2023_data.csv \
  /app/indexer/data/2024_data.csv \
  /app/indexer/data/2025_data.csv
```

Run once per CSV file. The default output name is <stem>_embedded.ndjson (e.g. 2025_data.csv → 2025_data_embedded.ndjson).

Option A — Inside Docker (CPU — slow for large datasets)

```
docker compose exec backend python indexer/export_embeddings.py \
        /app/indexer/data/2025_data.csv
```

Option B — On a GPU machine (recommended for full multi-year exports)
Run directly on a machine with a CUDA GPU. Install the dependencies first:

```
cd backend
pip install -r requirements.txt
# For NVIDIA GPU, install CUDA-enabled torch first:
# https://pytorch.org/get-started/locally/

python indexer/export_embeddings.py indexer/data/2025_data.csv

# Or with an explicit output path:
python indexer/export_embeddings.py indexer/data/2025_data.csv \
        --out indexer/data/2025_data_embedded.ndjson
```

Copy the resulting *_embedded.ndjson files into backend/indexer/data/ on the machine running OpenSearch, then continue with Step 5b.

Repeat for each year

```
docker compose exec backend python indexer/export_embeddings.py
/app/indexer/data/2020_data.csv
docker compose exec backend python indexer/export_embeddings.py
/app/indexer/data/2021_data.csv
# ... through 2025
```

5b. Import NDJSON into OpenSearch

> **This is the only supported way to load data for this project.** Do not run index_data.py on plain CSV for initial setup — documents will lack embedding vectors.

Single year

```
docker compose exec backend python indexer/import_embeddings.py \
    /app/indexer/data/2025_data_embedded.ndjson
```

All years

```
docker compose exec backend python indexer/import_embeddings.py \
  /app/indexer/data/2020_data_embedded.ndjson \
  /app/indexer/data/2021_data_embedded.ndjson \
  /app/indexer/data/2022_data_embedded.ndjson \
  /app/indexer/data/2023_data_embedded.ndjson \
  /app/indexer/data/2024_data_embedded.ndjson \
  /app/indexer/data/2025_data_embedded.ndjson
```

import_embeddings.py creates the index with k-NN enabled if it does not exist yet. Re-importing the same file is safe — documents are upserted by APPLICATION_ID, so no duplicates are created.

5c. Verify Data Loaded

```
curl http://localhost:9200/project_data/_count
```

The "count" value should match the number of records in your NDJSON file(s). To confirm vectors exist:

```
curl -s "http://localhost:9200/project_data/_search?size=1" | grep -o '"embedding"'
```

You should see "embedding" in the response.

6. Open the Application

Once services are running and data is loaded, open **http://localhost:5173** in your browser. Try a keyword search and the semantic/hybrid search to confirm everything is working.

Page	URL
Search	http://localhost:5173/search
Dashboard	http://localhost:5173/dashboard
Backend API docs (Swagger)	http://localhost:8000/docs
OpenSearch cluster health	http://localhost:9200/_cluster/health

7. Common Operations

Stop the Services

```
docker compose down
```

Indexed data is preserved in a Docker volume (opensearch-data). When you run docker compose up again, the data is still there — no need to re-import.

Stop and Delete All Data

```
docker compose down -v
```

The -v flag removes Docker volumes, which deletes the OpenSearch index. You will need to run Step 5b again (and Step 5a if you no longer have the NDJSON files).

Wipe and Rebuild the Index

Delete the index, then re-import embedded NDJSON:

```
curl -X DELETE "http://localhost:9200/project_data"
docker compose exec backend python indexer/import_embeddings.py \
    /app/indexer/data/2025_data_embedded.ndjson
```

Append a New Year of Data

Use append_data.py to add new data without wiping the existing index. The file path is passed as a positional argument:

```
# Append a new NDJSON file (recommended — preserves embeddings)
docker compose exec backend python indexer/import_embeddings.py \
    /app/indexer/data/2026_data_embedded.ndjson

# Or append a CSV directly (re-computes embeddings on the fly — slow on CPU)
docker compose exec backend python indexer/append_data.py \
        /app/indexer/data/2026_data.csv --with-embeddings
```

> **Note:** append_data.py takes a positional path argument — there is no --file flag.

Rebuild Just One Container

For example, after changing backend code:

```
docker compose up --build backend
```

Run Frontend Type-Checking

```
docker compose exec frontend npx tsc --noEmit
# or from the frontend/ directory on your host:
cd frontend && npx tsc --noEmit
```

8. Environment Variables Reference

All variables have safe defaults for local development. Set them in your shell or in a .env file before starting the backend container.

Variable	Default	Description
OPENSEARCH_HOST	localhost	Hostname or IP of the OpenSearch node
OPENSEARCH_PORT	9200	HTTP port for OpenSearch
OPENSEARCH_INDEX	project_data	Name of the OpenSearch index to create and query
CORS_ORIGINS	http://localhost:5173	Comma-separated list of allowed CORS origins for the API. Update this if the frontend runs on a different port — do not edit main.py directly
EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6-v2	HuggingFace model name for embeddings. Must match the model used at index time — changing after indexing requires a full reindex
EMBEDDING_DEVICE	auto	Device for embedding inference. auto selects CUDA → MPS (Apple Silicon) → CPU. Accepted values: auto, cuda, cuda:N, mps, cpu
EMBEDDING_ENCODE_BATCH_SIZE	32	Texts encoded per model.encode() call. Increase on GPU (e.g. 128) for faster throughput
EMBEDDING_ABSTRACT_MAX_CHARS	12000	Maximum characters of ABSTRACT_TEXT included in the embedding text. Set to 0 for no truncation

EMBEDDING_TERM_STATS_PATH	term_stats.json	Path to the precomputed term-frequency file from term_stats.py. Used to strip overly common terms before embedding
EMBEDDING_TERM_MAX_DF_RATIO	0.20	Document-frequency ratio above which a PROJECT_TERM is considered generic and stripped before embedding (e.g. 'research', 'data')
EMBEDDING_SHOW_PROGRESS	(unset)	Set to 1 to show a per-batch progress bar during indexing

9. Troubleshooting

OpenSearch keeps crashing or restarting

Cause: Not enough memory allocated to Docker Desktop.

Fix: Docker Desktop → Settings → Resources → Memory → set to 4 GB or more. Restart Docker Desktop.

curl http://localhost:9200 — connection refused

Cause: OpenSearch hasn't finished starting yet (takes 15–30 seconds).

Fix: Wait and try again. Check docker compose logs opensearch for errors.

Backend can't connect to OpenSearch

Cause: OpenSearch wasn't healthy when the backend tried to connect.

Fix: The Docker Compose health check handles this automatically. If it persists, run docker compose down && docker compose up --build.

Frontend shows network errors

Cause: Backend isn't running, or the frontend can't reach it.

Fix: Verify the backend is running at http://localhost:8000/health. The frontend expects the backend at http://localhost:8000 by default (set via the VITE_API_BASE_URL env var).

Search returns no results

Cause: Data hasn't been imported yet.

Fix: Complete Step 5b, then verify with curl http://localhost:9200/project_data/_count.

Semantic / vector search returns no results

Cause: Documents were indexed without embeddings (e.g. plain CSV via index_data.py without --with-embeddings), or NDJSON was never imported.

Fix: Delete the index and re-import embedded NDJSON (see Section 7 — Wipe and Rebuild the Index). Ensure each NDJSON line includes an embedding array.

'No stored vector' error on Similar Projects panel

Cause: That specific document was indexed without an embedding vector.

Fix: Re-import the NDJSON file that contains this document, or wipe and rebuild the index with embeddings.

export_embeddings.py is very slow in Docker

Cause: The default backend image runs the embedding model on CPU.

Fix: Run export_embeddings.py on a machine with a CUDA/MPS GPU (see Step 5a, Option B), then copy the *_embedded.ndjson files to the Docker machine and run import_embeddings.py in Docker.

CORS errors in the browser

Cause: The frontend is running on a port other than 5173.

Fix: Set the CORS_ORIGINS environment variable to include the correct frontend origin (e.g. CORS_ORIGINS=http://localhost:3000). Do not edit main.py directly.

npm ci fails in the frontend container

Cause: Stale node_modules volume.

Fix:

```
docker compose down -v
docker compose up --build
```

Empty filter dropdowns

Cause: The filter catalog endpoint (GET /search/filter-catalog) is called on mount. If the index is empty or OpenSearch is unreachable, the dropdowns will show no options.

Fix: Check the browser console for the underlying error. Verify OpenSearch is reachable and data has been loaded (Step 5b).